



Instituto Tecnológico de Mazatlán

Antipatrón del delay() y Temporización No Bloqueante con millis()

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Hernández Barrón

Alumnos:

Rosa Guadalupe Romero Navarro

Javier Ramón Quintero Valdez

Gabriel Alejandro Barrón Sarabia

Reporte de práctica: Antipatrón del delay() y Temporización No Bloqueante con millis()

1. Descripción del proyecto

Esta práctica se divide en dos partes que utilizan exactamente el mismo circuito físico (tres LEDs conectados a los pines 2, 3 y 4 del Arduino UNO R4 WiFi), pero con dos enfoques de programación distintos, con el fin de comparar sus resultados.

En la primera parte se controla el parpadeo de los tres LEDs utilizando delay(), función que detiene por completo la ejecución del microcontrolador durante el tiempo indicado. En la segunda parte, el mismo circuito se controla utilizando millis(), una función que permite verificar el paso del tiempo sin bloquear la ejecución del programa, logrando que cada LED parpadee de forma verdaderamente independiente.

2. Objetivos

- Identificar el comportamiento bloqueante de la función delay() en Arduino.
- Comprender por qué delay() impide la ejecución concurrente de múltiples tareas dentro de loop().
- Implementar una temporización no bloqueante utilizando millis().
- Comparar directamente ambos enfoques bajo el mismo circuito, para evidenciar sus diferencias.
- Aplicar el patrón de comparación de tiempos (ahora - previo >= intervalo) para controlar eventos periódicos.
- Extender el sistema con una tarea adicional independiente (mensaje por el Monitor Serial) sin afectar el comportamiento de los LEDs.

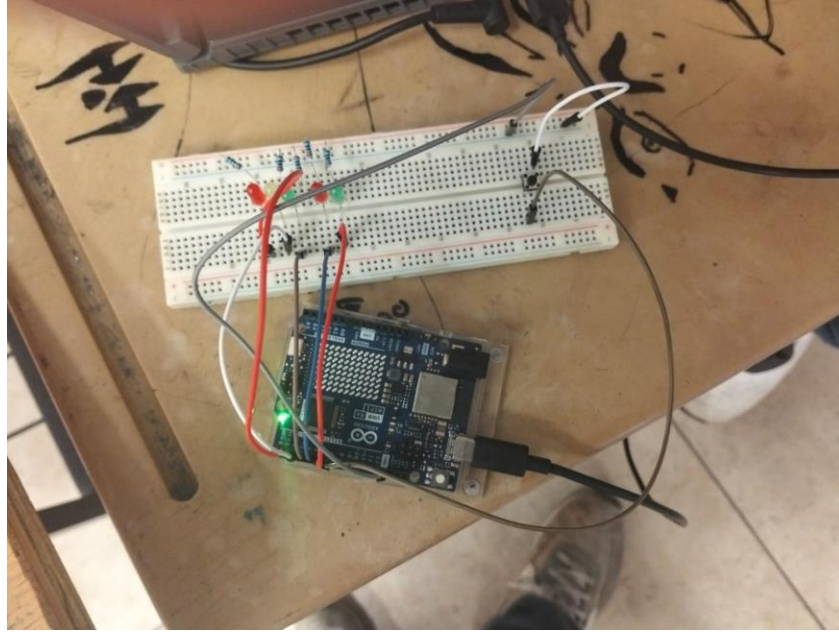
3. Herramientas y material utilizado

- Arduino UNO R4 WiFi
- Arduino IDE
- Protoboard
- 3 LEDs de colores distintos
- 3 resistencias limitadoras de corriente
- Cables de conexión (jumpers)
- Computadora

4. Diagrama

El diagrama representa la conexión utilizada para los tres LEDs, la cual se mantuvo idéntica para ambas partes de la práctica.

Cada LED se conecta en serie con su propia resistencia limitadora de corriente hacia GND. LED1 se conectó al pin digital 2, LED2 al pin digital 3 y LED3 al pin digital 4. No fue necesario modificar el circuito entre la Parte 1 y la Parte 2, únicamente el código.



5. Metodología

Primero se armó el circuito físico con los tres LEDs y sus respectivas resistencias, conectados a los pines 2, 3 y 4 del Arduino UNO R4 WiFi. Con este montaje ya listo, se desarrollaron los dos programas de forma independiente, manteniendo el mismo objetivo de parpadeo (LED1 cada 500 ms, LED2 cada 1000 ms y LED3 cada 1500 ms), pero con dos estrategias de temporización distintas.

En la primera versión (Parte 1), se programó cada LED utilizando `delay()` dentro de `loop()`, encendiendo y apagando cada uno de forma secuencial. En la segunda versión (Parte 2), se reescribió el programa utilizando `millis()`: para cada LED se definió una variable de estado (encendido/apagado) y una marca de tiempo del último cambio, comparando en cada vuelta del `loop()` si ya había transcurrido el intervalo correspondiente. Como reto adicional en esta segunda parte, se agregó una cuarta tarea independiente que imprime un mensaje por el Monitor Serial cada 3000 ms, sin compartir variables con la lógica de los LEDs.

Finalmente, se probaron ambas versiones por separado sobre el mismo circuito, observando el comportamiento de los LEDs y registrando evidencia mediante fotografías, video y capturas del Monitor Serial.

6. Explicación del funcionamiento

6.1 Parte 1 — `delay()` (antipatrón bloqueante)

```
digitalWrite(LED1, HIGH); delay(500); digitalWrite(LED1, LOW); delay(500);  
digitalWrite(LED2, HIGH); delay(1000); digitalWrite(LED2, LOW); delay(1000);  
digitalWrite(LED3, HIGH); delay(1500); digitalWrite(LED3, LOW); delay(1500);
```

La función `delay()` detiene por completo la ejecución del microcontrolador durante el tiempo indicado. Mientras el Arduino está "esperando" dentro de un `delay()`, no puede atender ninguna otra instrucción. Esto provoca que, aunque el código describe tres parpadeos con distintos tiempos, en la práctica el microcontrolador los ejecuta uno después del otro, de forma estrictamente secuencial: el LED2 no puede iniciar su ciclo hasta que el LED1 termine por completo el suyo, y lo mismo ocurre entre LED2 y LED3.

6.2 Parte 2 — `millis()` (temporización no bloqueante)

```
if (ahora - previoLED1 >= INTERVALO_LED1) {
```

```

previoLED1 = ahora;
estadoLED1 = !estadoLED1;
digitalWrite(LED1, estadoLED1);
}

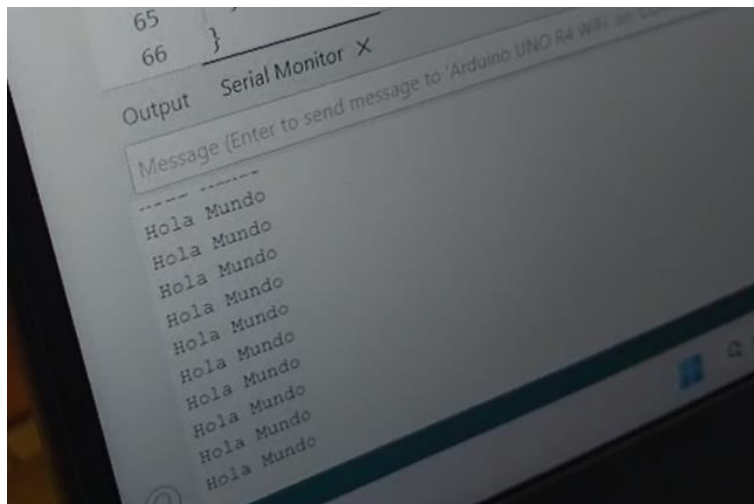
```

En cada vuelta del `loop()`, el programa obtiene el tiempo actual con `millis()` y lo compara contra la última vez que cada tarea se ejecutó. Si ya transcurrió el intervalo correspondiente, se actualiza la marca de tiempo y se invierte el estado del LED con el operador de negación (`!`). A diferencia de `delay()`, esta comparación no detiene la ejecución: el programa sigue evaluando las demás condiciones en la misma vuelta del ciclo, lo que permite que los tres LEDs, y la tarea adicional del mensaje serial, se ejecuten de forma prácticamente simultánea e independiente.

7. Análisis de resultados

Durante las pruebas de la Parte 1 se observó que los tres LEDs se encendían y apagaban uno a la vez, nunca simultáneamente entre sí. El tiempo real de repetición de cada LED correspondió a la suma de todos los `delay()` del programa ($500 + 500 + 1000 + 1000 + 1500 + 1500 = 6000$ ms), y no al tiempo individual programado para cada uno. Esto significa que, en la práctica, el LED1 no volvió a parpadear cada 500 ms como se pretendía originalmente, sino cada 6000 ms, una vez que los otros dos LEDs terminaban su propio ciclo.

En contraste, durante las pruebas de la Parte 2 los tres LEDs mostraron un parpadeo simultáneo, cada uno a su propia velocidad (500 ms, 1000 ms y 1500 ms respectivamente), algo que no era posible con `delay()`. Adicionalmente, el mensaje "Hola Mundo" se imprimió de forma consistente cada 3 segundos en el Monitor Serial, sin retrasos ni saltos, incluso mientras los LEDs cambiaban de estado, confirmando que la tarea del mensaje serial no interfirió con el parpadeo de los LEDs.



La comparación directa entre ambas partes deja en evidencia la diferencia fundamental entre los dos enfoques: mientras que `delay()` obliga a ejecutar las tareas de forma secuencial (sumando todos los tiempos de espera), `millis()` permite que cada tarea se ejecute de manera independiente y concurrente, sin bloquear la ejecución del resto del programa. No se observó ningún bloqueo perceptible en la Parte 2, y agregar la tarea del mensaje serial no requirió modificar la lógica ya existente de los LEDs, lo cual confirma que el patrón con `millis()` es fácilmente escalable.

8. Conclusiones

Gabriel

En esta práctica comprendí de forma muy clara por qué `delay()` es considerado un antipatrón cuando se necesita controlar varios eventos con temporizaciones distintas. Al comparar ambas versiones del código sobre el mismo

circuito, pude ver con mis propios ojos cómo un programa que en papel parecía hacer tres tareas al mismo tiempo, en realidad las ejecutaba una por una, y cómo `millis()` resuelve ese problema sin necesidad de detener el programa.

Javier

Durante esta práctica aprendí a aplicar el patrón de comparación de tiempos con `millis()`, separando para cada LED su propia variable de estado y su propia marca de tiempo. Lo que más me quedó claro fue la diferencia entre "esperar" con `delay()` y "verificar si ya pasó el tiempo" con `millis()`: esta segunda forma permite que el microcontrolador siga revisando otras condiciones mientras tanto, en lugar de quedarse detenido.

Rosa

Esta práctica me permitió comprender cómo realizar varias tareas al mismo tiempo utilizando una programación no bloqueante. Ver el mismo circuito comportarse de manera completamente distinta solo por cambiar `delay()` a `millis()` me ayudó a entender por qué esta técnica es tan importante en proyectos donde se necesita que varias cosas sucedan sin que una bloquee a la otra, como en este caso los LEDs y el mensaje del Monitor Serial ejecutándose en paralelo.